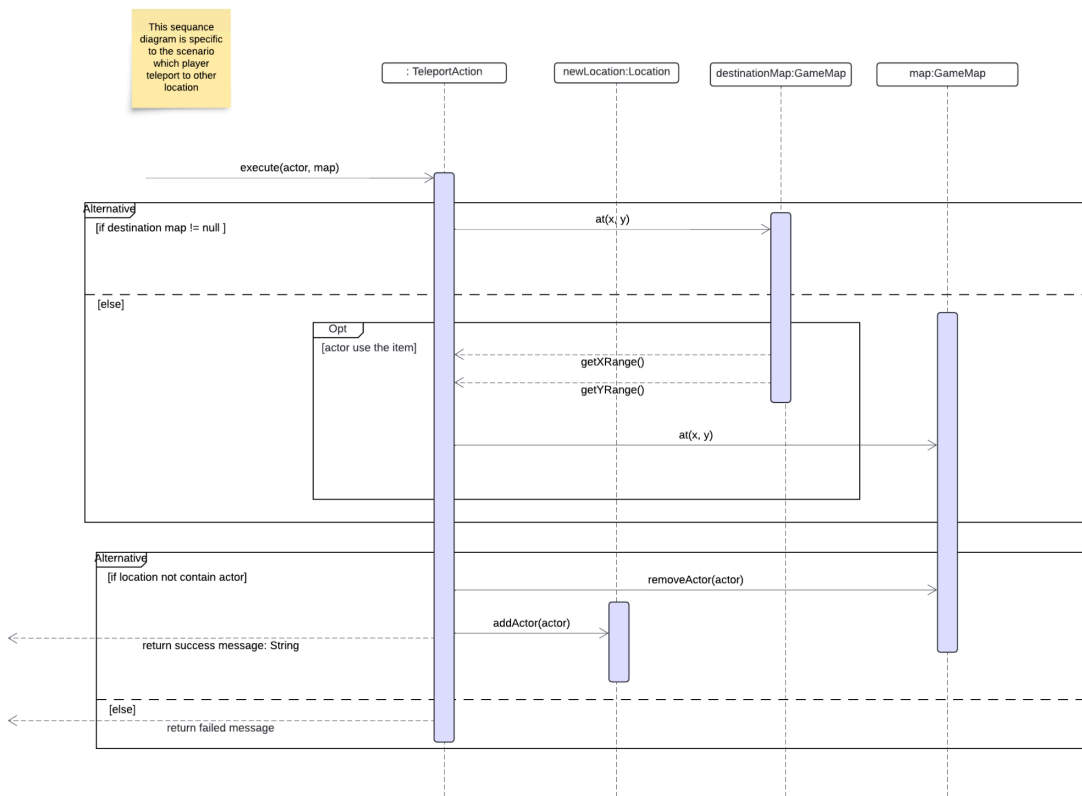
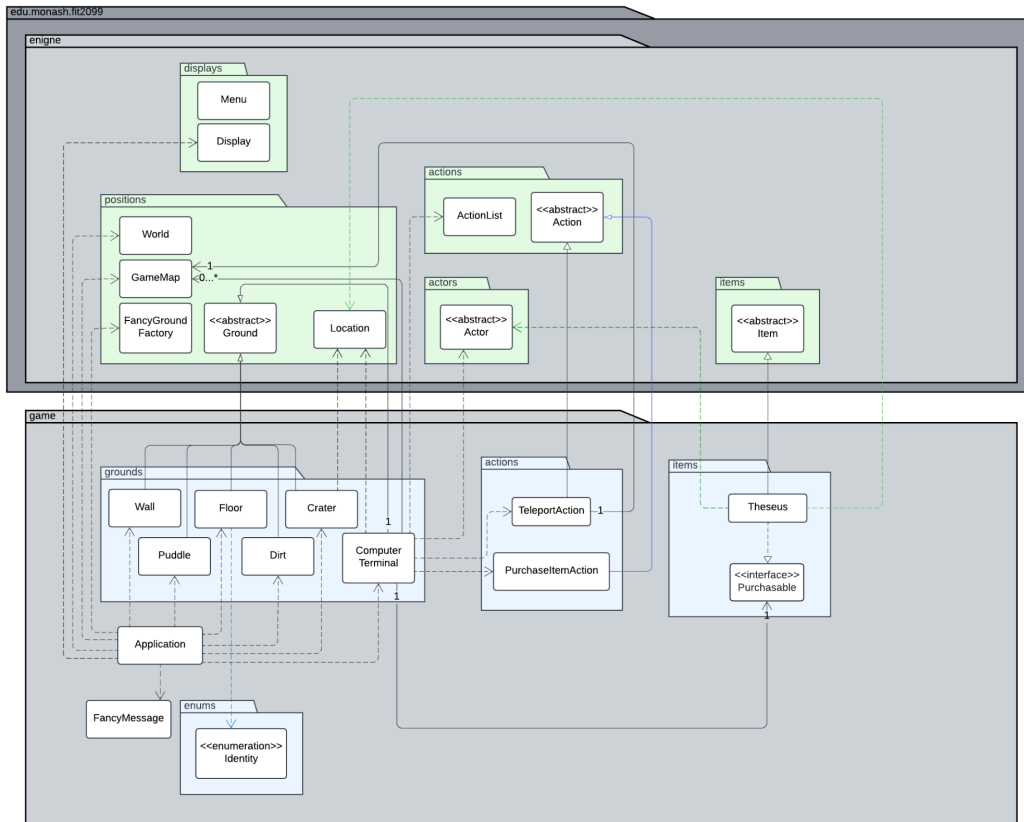


Assignment 3 Design Rationale - Applied 29 grp 3

Requirement 1: The Ship of Theseus



New maps are added in the Application class with the manually set of teleport destinations and computer terminals on the new maps. The Theseus and Teleport Action class are created to handle teleportation. The computer terminal class added allowable action of teleportation.

Theseus

The Theseus class represents a portable teleporter item that can be purchased and used by the intern to teleport randomly within the current map.

The Theseus class manages properties such as its name, display character, and price, while also handling behaviours like purchasing and a specific teleport action. This aligns with the Single Responsibility Principle (SRP) ensures that each aspect of the Theseus item is handled independently, making the code more organised, understandable, and easier to maintain. It also promotes the Open/Closed Principle (OCP) by allowing for extension without modification. The Theseus class can be extended to include additional functionalities or actions without changing the existing code.

The advantages are easier maintenance as it has focused responsibility. However, If complex functionalities are added directly, it could violate SRP and make the class harder to maintain.

Teleport Action

The Teleport Action class represents an action that executes teleportation within the game maps. It calculates the destination location and handles the movement of the actor accordingly.

It handles teleportation action execution separately from other concerns, allowing changes to the teleportation mechanism without affecting other parts of the code adhering to the Single Responsibility Principle (SRP). The Open Closed Principle (OCP) is achieved by allowing new teleportation types to be added through subclassing, while the DRY principle is maintained by centralising the teleportation logic.

The advantages are easily extensible for new teleportation behaviours and simplifies maintenance due to centralised teleportation logic. However, there is limited flexibility for more complex teleportation rules.

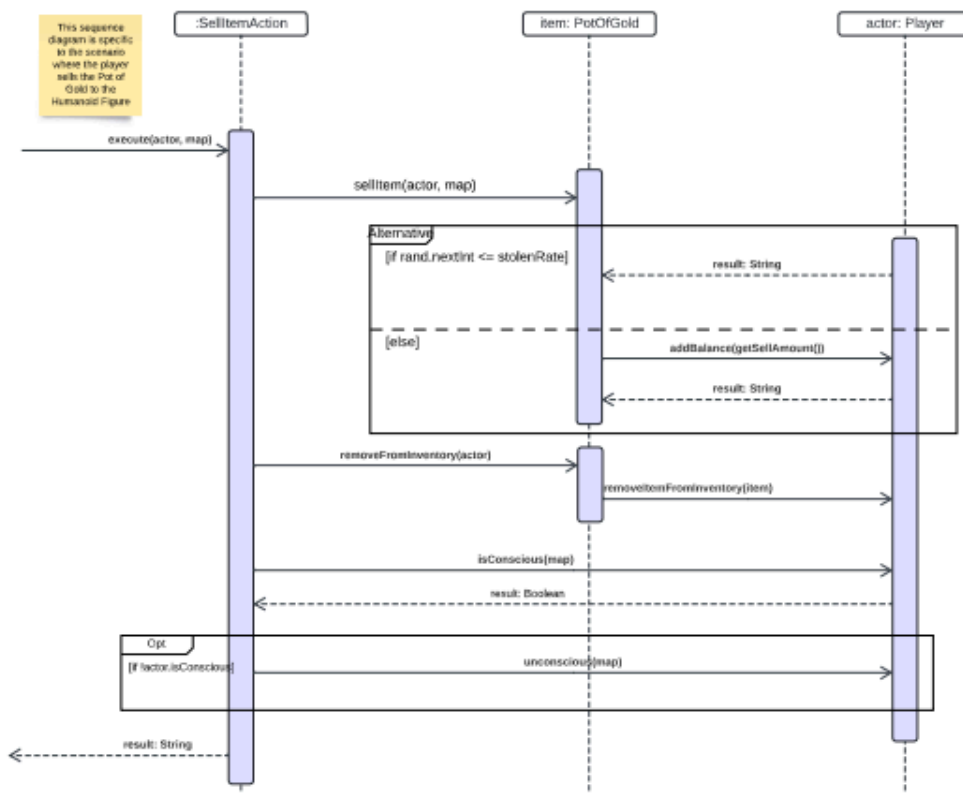
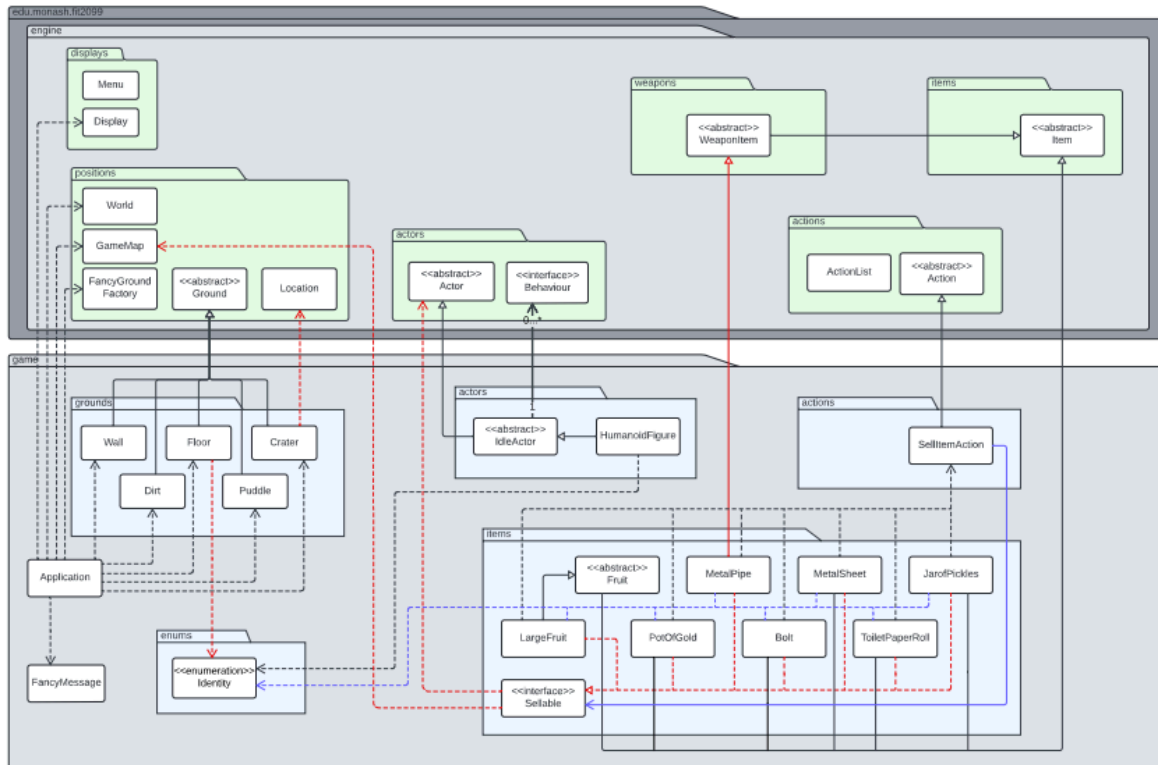
Computer Terminal

The computer Terminal class allows players to interact with to purchase items and teleport between maps.

The ComputerTerminal is only responsible for displaying available items, while the PurchaseItemAction class handles purchase logic and TeleportAction class handles teleportation logic. This adheres to the Single Responsibility Principle (SRP).

This design choice simplifies debugging and future modifications but may require additional classes and potential complexity if many interactions are added.

Requirement 2: The Static Factory



This task will require the player to be able to sell their items from their inventory to a new actor 'Humanoid Figure'. Each item has their unique perks or conditions when attempting to

sell, as well as different item sell values. The player can only sell items when they are in the vicinity of Humanoid Figure.

Sellable

This is an interface to group up items that can be sold and retrieve their respective sell value. The implementation of this interface aligns with the Dependency Inversion Principle (DIP), which promotes decoupling and easier maintenance, as all sellable items have their methods defined in this interface. It also achieves Open-Closed Principle (OCP) by utilising this interface, which allows for future extensions without ever modifying the original code.

An alternative approach would be using an abstract class instead of an interface, which works well on paper. However, multiple inheritance for a class would prove to be challenging in terms of readability and maintenance, as well as leading to unnecessary complexities. Not only that, java also does not support multiple inheritance.

SellItemAction

To handle actions of the player, specifically selling items in this case, this SellItemAction class will provide the menu description as well as handle the execution of user input if the player decides to sell an item.

Action classes like this are proven to be useful in handling specific actions, while unaffected logic from other sources. In other words, it adheres to the Single Responsibility Principle (SRP) as it only handles the selling of an item. It also supports the Dependency Inversion Principle (DIP) as it is basically extended from the Action abstract class. The main benefit for implementing actions that can be done by the player in this way, is that whenever there are new actions to be added, we would not need to modify any of the original code.

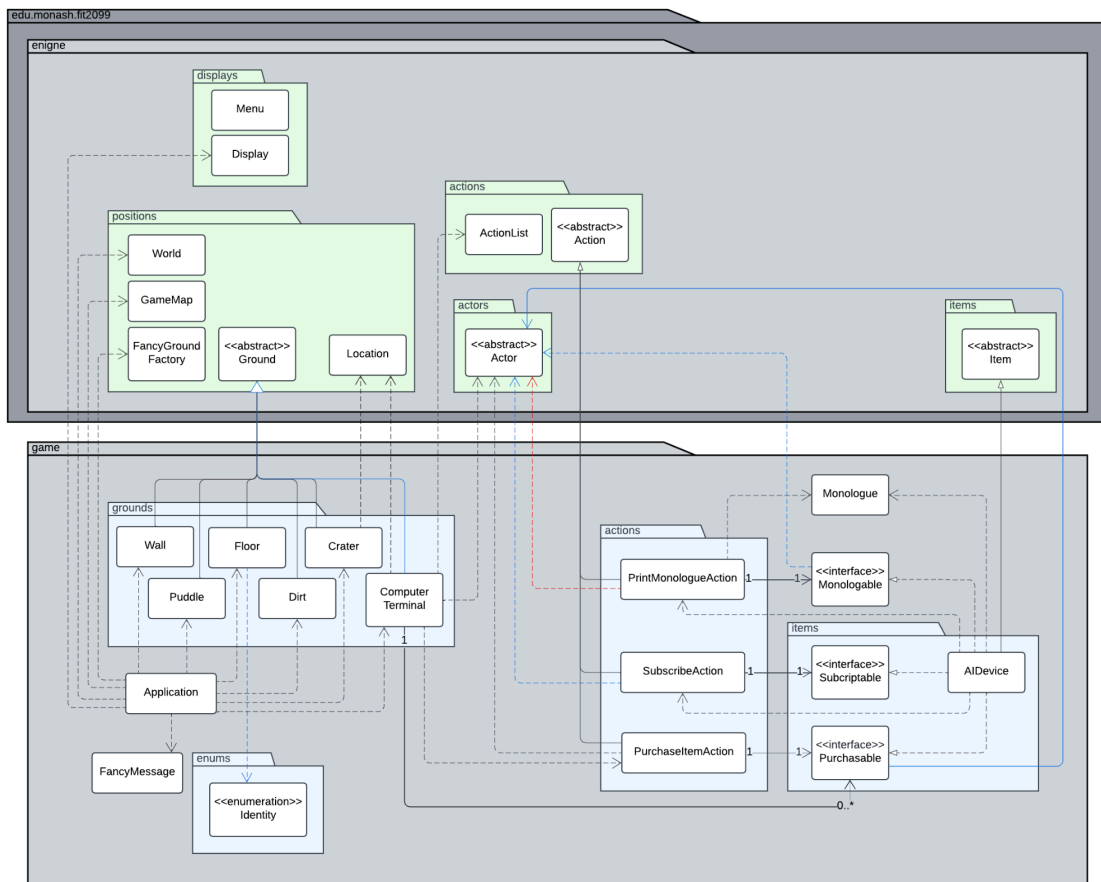
IdleActor (abstract) and HumanoidFigure

Since there are multiple actors of different behaviours (some can only attack while some can only wander, etc.), it would be easier for us to use abstractions for the actors, allowing easier future modifications. For actors that cannot move but have the potential to attack, the IdleActor abstract class, which extends from Actor abstract class, is implemented, then the HumanoidFigure would extend from it as one of the 'IdleActors' in the game.

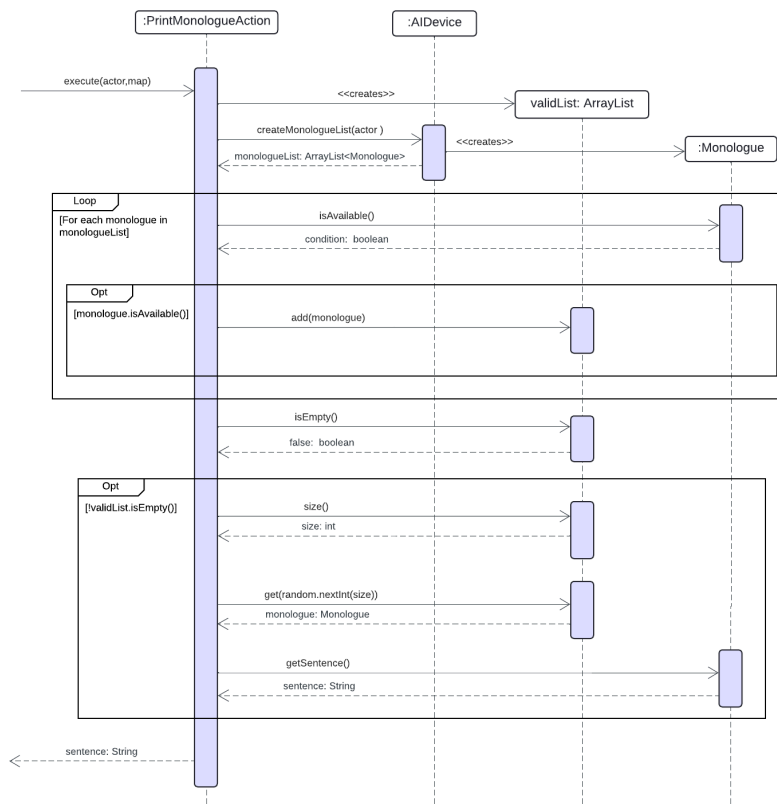
Both of these classes adheres to the Liskov Substitution Principle (LSP) by ensuring that they can replace their objects of their parent class without changing the original functionality. As this approach respects the benefits of abstraction, it also strongly follows the Open-Closed Principle (OCP), such that new actors that share some of the behaviours can extend from IdleActor, with their own unique behaviours, without ever affecting the source.

An alternate approach would be directly extending new objects from the Actor class, which avoids the need for creating another abstract class to avoid overengineering, in this case IdleActor. Unfortunately with this approach, we would have to implement the same functionalities for each new actor if they ever share the same behaviours, thus violating Don't Repeat Yourself (DRY). Abstractions would also provide better arrangements of classes for easier reviews or future modifications.

Requirement 3: dQw4w9WgXcQ



This sequence diagram is specific to the scenario where the player listen to the AI Device and a monologue is printed out



This task requires the player to be able to purchase an AI device called Astley for 50 credits from the computer terminal. The player can listen to the AI device's monologue, which is chosen randomly from a predefined list. Every 5 ticks, the player has to pay the subscription fee or the device becomes unusable until the intern has sufficient credits.

AIDevice

The AIDevice class represents the AI device that intern can purchase and interact with as it implements the Purchasable, Monologable and Subscriptable interfaces. The implementation of this class adheres to the Single Responsibility Principle (SRP) as it is responsible for handling the functionalities specific to the AI device, such as subscription management and providing monologue options. It also aligns with the Open-Closed Principle (OCP), since we can easily add new features without modifying the existing code.

Subscriptable and SubscribeAction

The Subscriptable interface is designed to represent items that require a subscription in the game. It allows for different items to be subscribed to by defining methods for subscription fees, subscription descriptions and subscription status. Meanwhile, the SubscribeAction class is responsible for checking if the actor has enough credits to subscribe and handling the subscription process. This class adheres to the Single Responsibility Principle (SRP) by only handling the subscription logic and leaving other responsibilities to other classes. It also supports the Dependency Inversion Principle (DIP) by depending on the Subscriptable interface rather than concrete implementation, enabling polymorphism.

Monologue

The Monologue class represents individual monologue entries. Each monologue has a sentence and a condition that determines its availability. This class achieves the Single Responsibility Principle (SRP) as it only handles the data and logic related to a monologue. It also adheres to the Open-Closed Principle (OCP) as new monologues can be easily added to the actors/items/grounds/ that can monologue without changing the existing monologues.

Monologable and PrintMonologueAction

The Monologable interface is designed for items, actors or grounds that can have a monologue while the PrintMonologueAction class handles the action of printing a monologue. This interface ensures that any class that can monologue will provide the necessary method to create its monologue list, promoting consistency and reusability. Meanwhile, the PrintMonologueAction class encapsulates the logic for printing monologues, making it easy to manage and extend.

This implementation achieves the Dependency Inversion Principle (DIP) since high-level modules depend on the abstraction (Monologable interface) rather than concrete implementation, promoting decoupling. Beside, the PrintMonologueAction class adheres to the Single Responsibility Principle (SRP) since it is responsible only for executing the print monologue action. It also supports the Open-Closed Principle (OCP) as new types of monologable entities can be added without modifying this class, as long as they implement the Monologable interface.

Additionally, the implementation of the Subscriptable interface and Monologable interface also achieves Interface Segregation Principle (ISP) as each interface only includes methods that they care about. For example, the Subscriptable interface only has methods related to the subscribe process such as `getSubscriptionFee()` and `isSubscribed()`, while the Monologable interface only includes the `createMonologueList()` method.

Benefits

In this design each class and interface has a single, well-defined responsibility. Furthermore, new functionalities, such as additional monologable entities or new monologue conditions can be added with minimal changes to existing code, ensuring extensibility. The use of interfaces also promotes reusability across the system.

Limitations

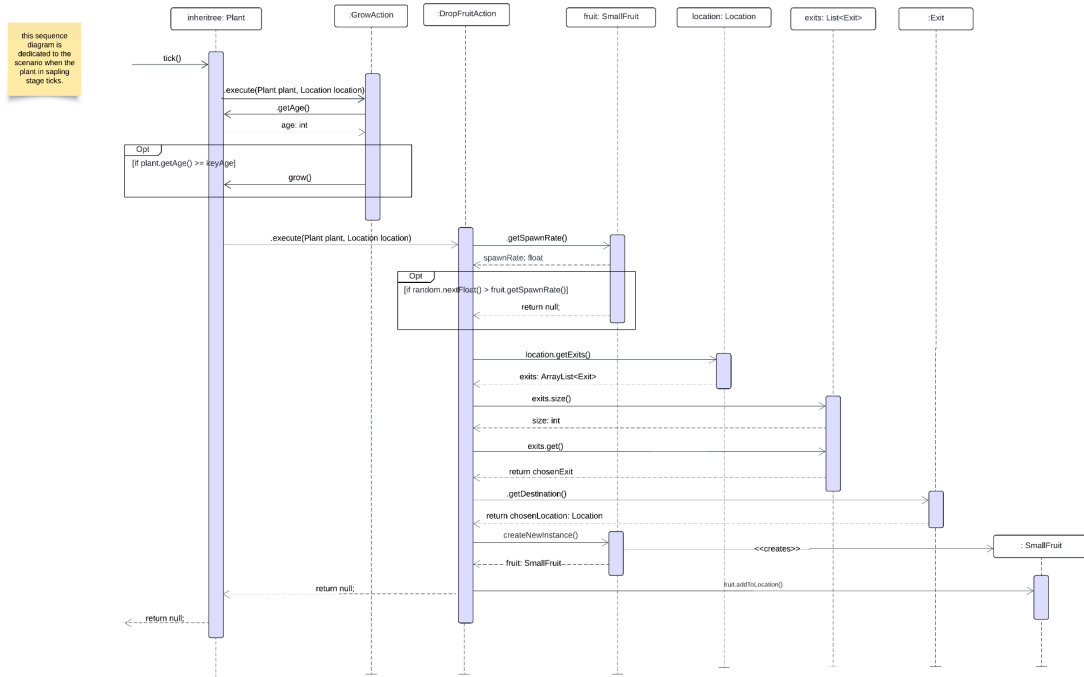
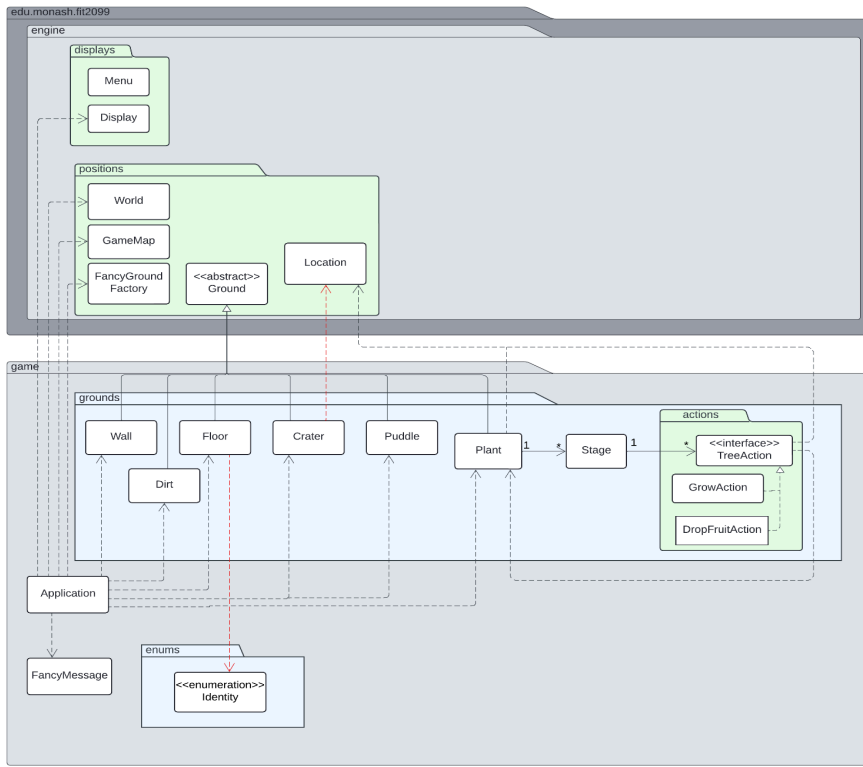
This design introduces multiple classes and interfaces, which can increase complexity.

Alternative Implementation

An alternative approach could involve using an abstract class for Monologable rather than an interface, allowing for shared implementation details. However, this approach was not chosen because Java does not support multiple inheritance, limiting the flexibility of using abstract classes for entities that might need to inherit from other classes. In this case, the AI device needs to inherit from the Item class, thus not suitable for using abstract classes for Monologable.

Requirement 4: Refactorio, Connascence's largest moon

Req 4



Refactoring made:

- Plant changed from abstract class to concrete class
- Inheritree class is removed
- TreeAction interface is added
- Removed Stage's association to Fruit and changed to TreeAction instead

Plant and Stage

Plant is a class that extends from Ground and represents a typical plant in the game. It has a list of Stages, arranged in the same order as in its actual lifecycle. In each stage, it has a list of TreeAction that the plant can do in that stage and stores information about its display character and an integer representing how many ticks needed to reach the next stage.

Plant contains some getters, setters, grow() function and overridden tick(). Grow function is called by GrowAction, so technically it is not handling the grow logic. It is just a function that the other class can interact with, just like Actor's hurt() function. The grow function will move onto the next stage in its list and update its display character.

In each tick, it will execute every TreeAction that the current stage is able to do. These actions are totally automatic, do not require any interaction with other actors, it is obviously not a typical Action. Hence, a TreeAction interface is made just for these Actions.

Plant is made a concrete class and Inheritree is removed because currently the Plant class can handle all the base logic of a plant and inheritree does not have characteristics special enough to make it a new class.

Benefits:

- It aligns with SRP as the plant does not need to handle every stage now, it just has to contain information about the Stages (the list).
- Adding new Stages into a plant's life cycle just requires a few lines of code in the Application, there is no need to modify any existing code. If there is any plant with more dynamic behaviour or characteristics appearing in the future, they can simply extend the Plant class without the need to repeat the lifecycle handling. This aligns with the Open/Closed Principle (OCP) and Don't Repeat Yourself (DRY).
- Managing all the stages using a list will greatly improve the maintainability and extensibility of the code, as all the stages are arranged in the same order as the lifecycle.

Connascence Analysis between Plant and Stage

- There are association between Plant and Stage class
- Connascence of Name - Plant called several functions of the Stage class
- Connascence of Type - The return type of the functions called should be correct
- Connascence of Order - The stages need to be arranged correctly for the plant to grow correctly.
- Connascence of Identity - The plant depends on the stage to store information about every stage, so the exact stage instance is needed to get the correct information.
- Due to the strong dependency of Plant on Stage, there is a tight coupling between Stage and Plant. Making changes to Stage will lead to modification of Plant class. However, the strong coupling is inevitable as stage is an inherent part of Plant.

Removing the coupling will be at the cost of violating several SOLID principles and lead to code with worse quality.

Alternative implementations

1. Make a class for every stage in the life cycle

- When more and more stages or plants are added, there will be too many classes to handle. It will greatly reduce the maintainability and code quality.
- Furthermore, it will still suffer from the Connascence of Order as the lifecycle chain needs to be correctly linked.
- each stage is actually really similar, it will probably violate the DRY.

2. Remove Stage and make several Plant instances for every stage in the life cycle and directly pass-in the TreeAction in Application. This is similar to my current implementation and might seem simpler at first glance. However, there is an explicit link between each GrowAction to the next Plant instance in the life cycle.

- This results in a chain of loose coupling, where any changes to the current life cycle will require some modification to the code. It will also violate OCP and greatly reduce maintainability.
- In my implementation, I removed the chain of coupling by using the Stage class which will be managed using an ArrayList. These stages do not have a link to another class, adding or removing a stage from the life cycle does not involve modification to unrelated code.

TreeAction

TreeAction is an interface dedicated to actions that a Plant can and must do in each tick. It only consists of one function - execute(Plant plant, GameMap map).

Limitation:

TreeAction can only be used by Plant and its future children, the best way is to have a General Action dedicated to Ground. However, due to the complexity of a Tree with its unique behaviour, it is reasonable to have an interface just for it. It does not violate LSP, but not the most ideal way of doing it.

Benefits:

- TreeAction efficiently encapsulates all the additional logic of growing and dropping fruit to individual TreeAction classes. This aligns with ISP as it makes sure that only necessary logic is included and removes redundancy.
- TreeAction creates a template of how a Tree Action class would behave, so that Plant can just store a list of TreeAction and interact with each action without needing to worry about the exact implementation. (DIP)
- Each TreeAction class is only responsible for one task only, it aligns with SRP and keeps our class focused.
- TreeAction classes are easily reusable by other plants with similar behaviour, this will reduce redundancy in the code. (DRY)
- When the plant has more features in the future, they can just create a new TreeAction to handle them without needing to change any existing code (OCP).

Connascence analysis between Plant, Stage and TreeAction

- Although stage has association with TreeAction, but stage does not use TreeAction in its class, so there is a loose coupling between them.
- There is a dependency between TreeAction and Plant. Plant will call TreeAction's execute function in tick and this will lead to Connascence of Name.
- TreeAction will take in the Plant that executes the action and the plant location as parameter and it leads to Connascence of Type and Position.
- There is only one method involved and the order in which the action is executed does not matter, there is no Connascence of Execution.
- There are no magic numbers or semantically linked values, hence there is no Connascence of Meaning and Value.
- Despite the dependency, the connascence between Plant and TreeAction are all static, meaning they are quite weak. There is a loose coupling between Plant and TreeAction and between Stage and TreeAction.
- The coupling between these three classes is reasonable and shouldn't be a problem as it results in better code quality.

Alternative implementations

1. The plant class will handle some of the tree behaviour logic and leave some to its child class. For example, Plant will handle the growing and Inheritree will handle the fruit-bearing.

- In some stages of the inheritree's life cycle, it will not drop fruit, assuming the inheritree will drop fruit as a whole will violate ISP.
- Each class will handle too much responsibility and make them a god class, it will violate SRP and reduce code quality.
- It might remove the dependency to TreeAction but at the cost of reduction in code quality.